

CS3.304: Advanced Operating Systems

(L-T-P: 3-1-0 Credits: 4)

P. Krishna Reddy

E-mail: pkreddy@iiit.ac.in

<http://www.iiit.ac.in/~pkreddy>

Office: Data Sciences and Analytics Center (DSAC)



Prof. P. Krishna Reddy
Data Science and Analytics Center, IIIT Hyderabad, India
E-mail: pkreddy@iiit.ac.in; <http://www.iiit.ac.in/~pkreddy>
Phone: 040-66531322; +91-9849329324



Research Areas

•Data mining/ Web Mining/ Bio-mining

Diverse frequent patterns, Coverage Patterns, Periodic frequent patterns, Rare knowledge extraction, Classification/prediction, Clustering.

Internet monetization, Search engine advertisement, Banner advertisement placement, Recommendation systems, Link-based community extraction, E-commerce, Information extraction

Protein function prediction.

•Database Systems/ Systems Building

Schema summarization, Transaction synchronization, Database recovery, Data replication, Speculative transaction management, Memory efficient mining, User interfaces, Weather-based decision support systems.

•IT for Agriculture

Personalized agro-advice advisory, Knowledge transfer, Systems for bridging lab to land gap, Virtual crop labs for agriculture education, Location-specific content development, Agriculture extension, Agriculture planning tools for a farmer.

•IT for law:

Finding similar judgements

Systems Built

- eSagu: An IT-based Personalized Agro-Advisory System
- eAquaSagu: An IT-based Personalized Aqua-Advisory System
- eAgromet: An IT-based agro-meteorological advisory system
- Village-level eSagu: A Scalable and Location-Specific Agro-advisory System

Key Ideas Developed

- Coverage patterns, Diverse Patterns
- Dense-bipartite graph based web communities
- Rare frequent /knowledge patterns.
- Periodic frequent patterns
- Weather condition, Coupled weather condition
- Deadlock prevention using data flow graphs.
- Replica synchronization using data flow graphs
- Backup commit protocol
- Speculative locking protocol for regular and read-only transactions
- Temporality-based user interface design
- Query by object based user interface design
- Personalized agro-advisory (eSagu and eAgromet)
- Post-graduate Diploma in applied agriculture and IT (PGDAAIT)
- Virtual crop labs.

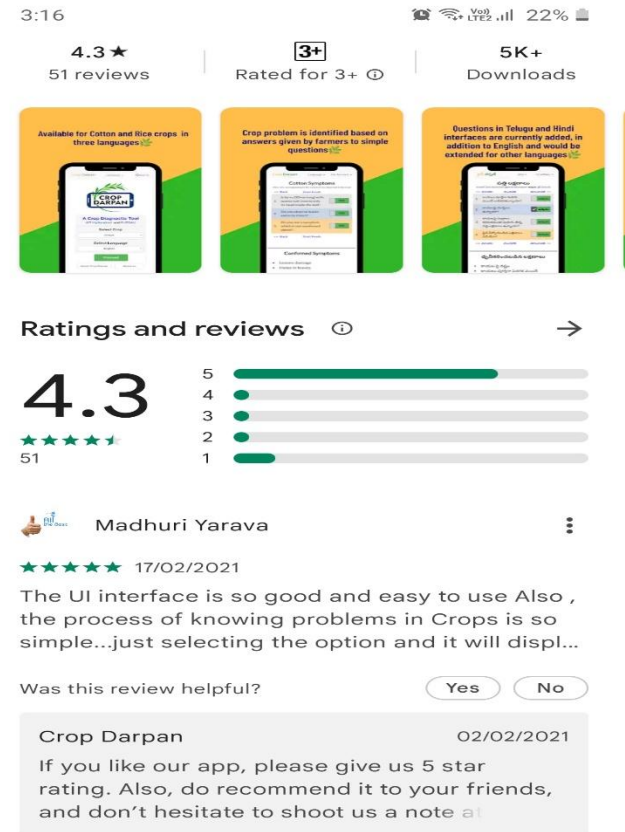
Future Research

- Diverse frequent patterns based recommendation and search systems.
- Coverage patterns based banner/search engine advertisement management
- Periodic frequent pattern based knowledge extraction system
- Efficient knowledge transfer systems
- Coverage pattern based subgraph discovery and visual mining
- Building query by object interface for non-SQL users
- Speculation-based synchronization for big data
- Protein function prediction
- Scalable decision support system for agriculture
- Building of resource planning tool for farmers
- Virtual crop labs for enhancing practical agricultural education.
- Decision support system for law

Farm diagnostic tool

Install Crop Darpan App in
your smart phone

- For desktop, the system is available at www.cropdarpan.com
- Crop Darpan App is available (on Andoid OS) at play store. Download and give feedback.
<https://play.google.com/store/apps/details?id=in.iiit.cropdarpan>
- App is live on Apple App Store.
<https://apps.apple.com/us/app/crop-darpan/id1556486922#?platform=iphone>



Install Crop Darpan in your smart phone

- For desktop, the system is available at <www.cropdarpan.com>
- Crop Darpan App is available (on Andoid OS) at play store. Download and give feedback.

<https://play.google.com/store/apps/details?id=in.iiit.cropdarpan>

- App is live on Apple App Store.
<https://apps.apple.com/us/app/crop-darpan/id1556486922#?platform=iphone>

Courses You Have Completed

- Digital logic design
 - Build an integrated circuit for a given function.
- Computer Architecture
 - Develop an assembly language program given a problem
- Programming language
 - Develop a high level program to solve a given problem.
 - printf()
 - scanf()
 - fopen()
 -

Outline

- **Introduction**
 - **What is an Operating System ?**
- Coping with the complexity
- Course topics and grading

Questions

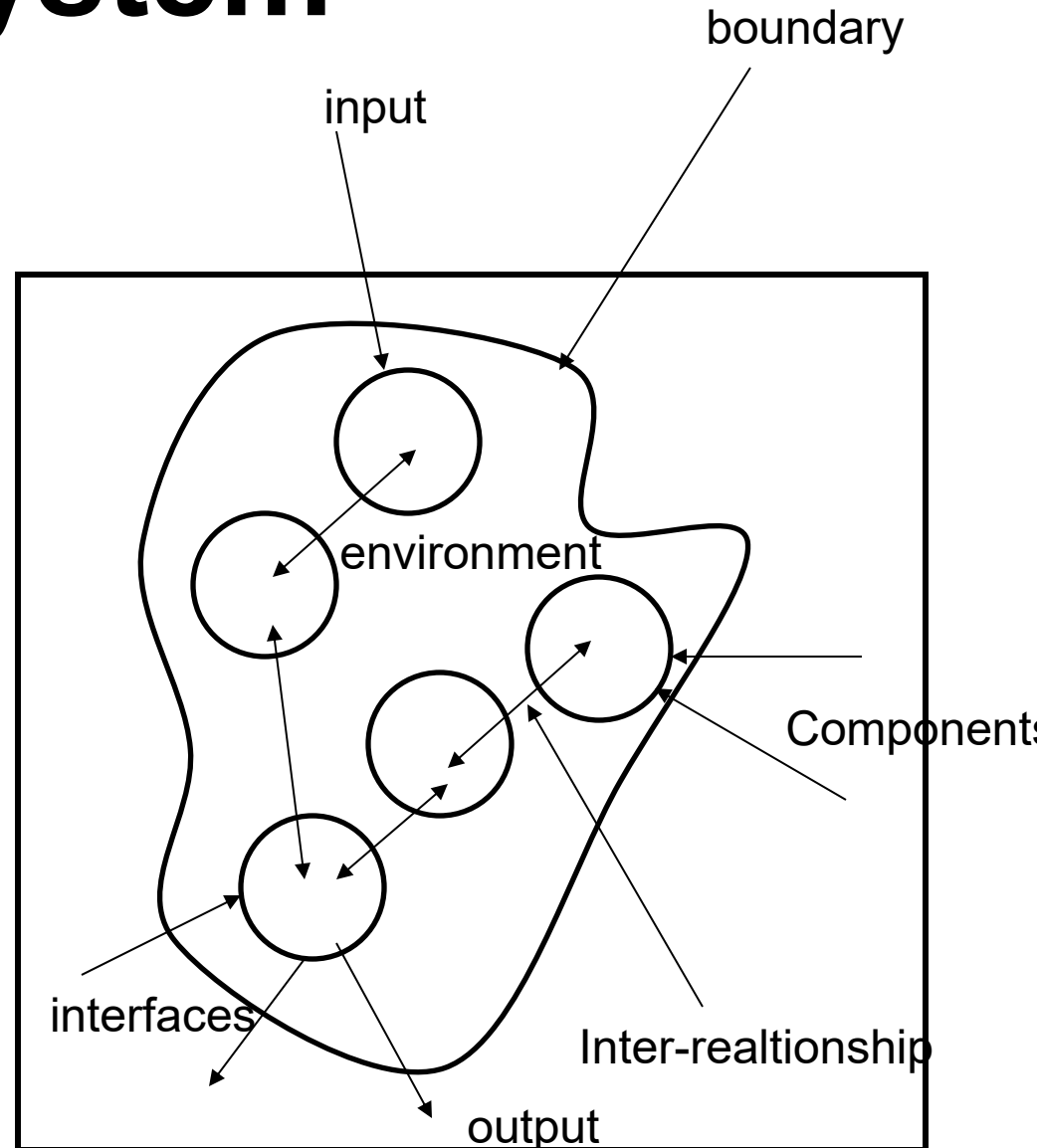
- What is a system ?
- What is an operating system ?
- What is a computer operating system ?

What is a system ?

- A system is an inter-related set of components with an identifiable boundary working together for some purpose.
- System can be natural or fabricated
 - Natural systems: human body or solar system
 - Fabricated systems: cycle, bus, computer, government, boat

System

- A system has nine characteristics
 - Components
 - Inter-related components
 - A boundary
 - A purpose
 - An environment
 - Interfaces
 - Input
 - Output
 - Constraints.



A general depiction of a system

Characteristics...

- Components:
 - A system is made up of components
 - A component is either an irreducible part or aggregation of parts that make-up a system. A component is also called a sub-system.
- Interrelated:
 - The components of interrelated
 - Dependence of one subsystem on one or more subsystems.

Characteristics...

- Boundary (Scope):
 - A system has a boundary, within which all of its components are contained and which establishes the limits of a separating the system from other systems.
- Purpose
 - The overall goal of function of a system.
The system's reason for existing.

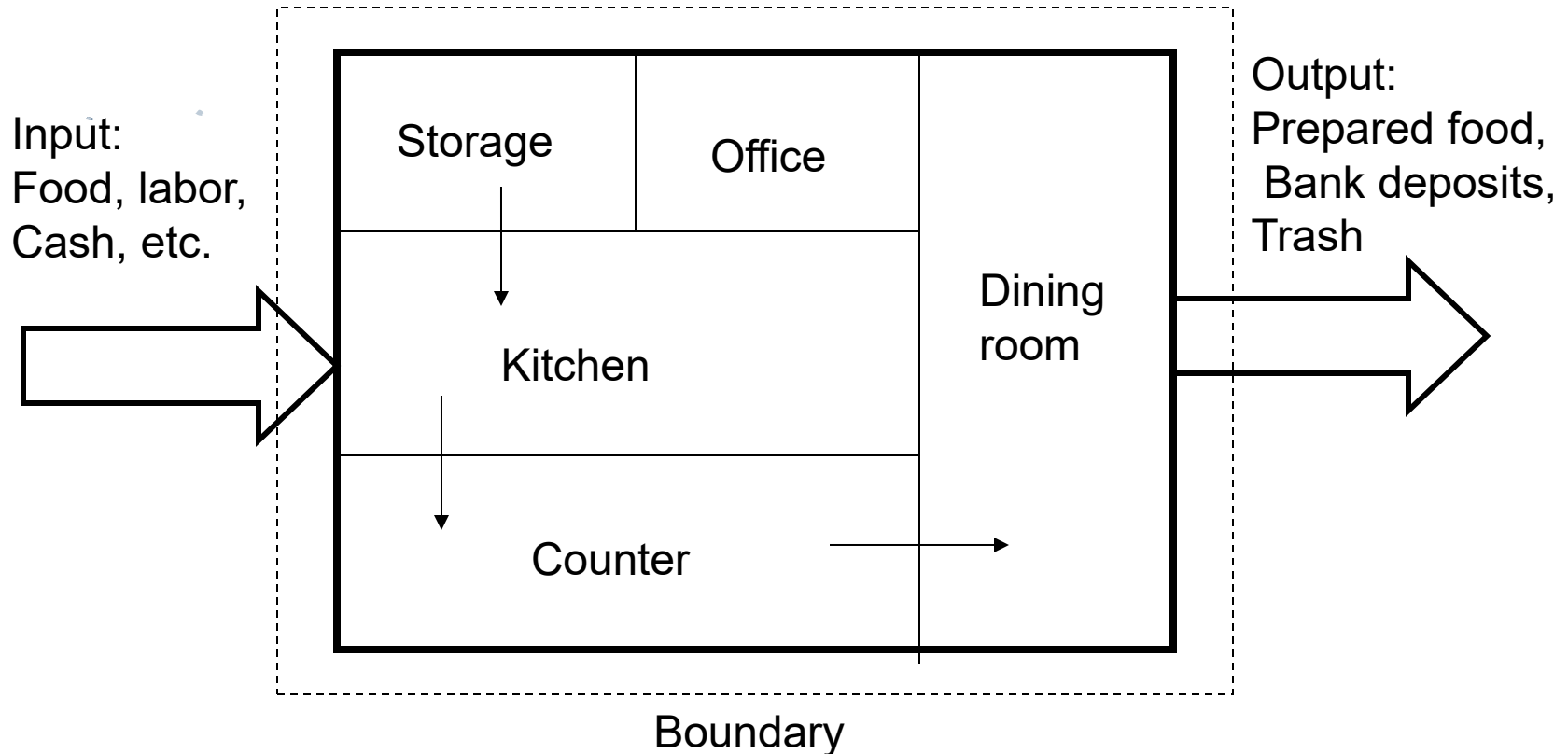
Characteristics...

- Environment
 - Everything external to the system that interacts with the system.
- Interface
 - Point of contact where a system meets its environment or subsystems meet each other.
- Constraint:
 - A limit what a system can accomplish: Capacity, speed or capabilities.

Characteristics...

- Input
 - Whatever a system takes from its environment in order to fulfill its purpose.
- Output:
 - Whatever a system returns to its environment in order to fulfill its purpose.

Example: A fast food restaurant as a system



Environment: Customers, food distributors, banks etc

—————> Represents an inter-relationship

Constraints: Popular foods, Health dept., constraints of storage

What is an operating system ?

- Operating system is a system.
- Operating system is a subsystem of any tool.
- Each tool constitutes machine part and operating part.
- **The operating part of a tool is called as operating system of that tool.**
- The purpose of operating system is to facilitate the operation of the underlying machine or tool.
- For some tools, operating system may not exist.
 - Example: Pen.
- For a user, the operating system abstracts the machine part in terms of simple services by hiding the details of the machine. The OS can provide services to users or other subsystems.
- Examples of typical operating systems:
 - Car operating system, Telephone operating system, TV operating system and so on.

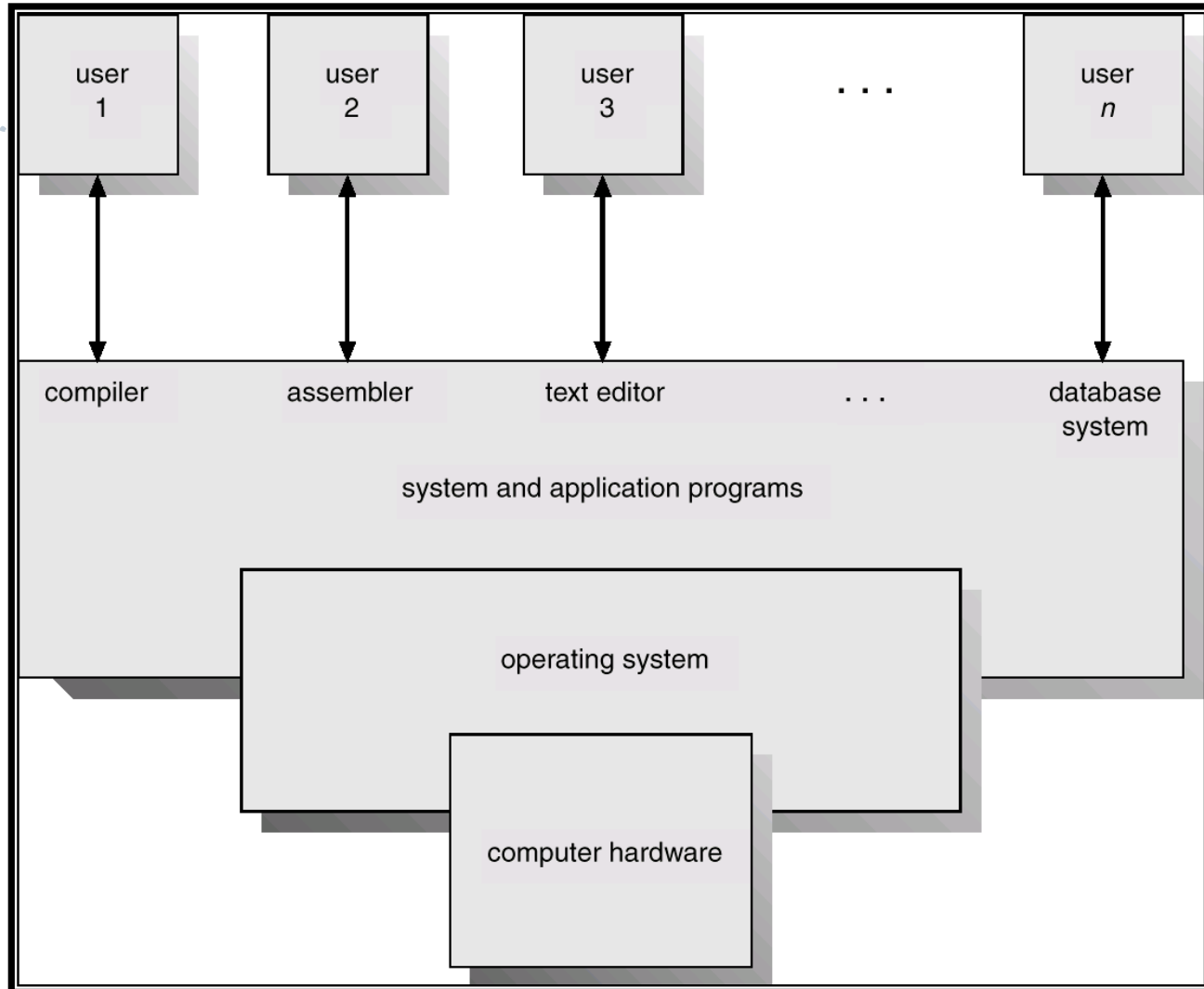
What is a computer operating system ?

- A computer is also a tool that contains the machine part and the operating part.
- The operating part of a computer is called Computer Operating System.
 - The operating system abstracts the underlying hardware in terms of simple services by hiding the details of the hardware. The OS can provide services to users or other subsystems.
- Examples of Computer operating systems:
 - WINDOWS 10, Macintosh, UNIX, SOLARIS, LINUX, Android, MAC IOS and so on.
- In the rest of this course, operating system means computer operating system.

Computer System Components

1. **Hardware** – provides basic computing resources (CPU, memory, I/O devices).
2. **Operating system** – controls and coordinates the use of the hardware among the various application programs for the various users.
3. **Applications programs** – define the ways in which the system resources are used to solve the computing problems of the users (compilers, database systems, video games, business programs), Internet.
4. **Users** (people, machines, other computers).

Abstract View of System Components



What is an Operating System?...

- A program that acts as an intermediary between a user of a computer and the computer hardware.
- Operating system goals:
 - Make the computer system convenient to use.
 - Use the computer hardware in an efficient manner.

Operating System Definitions...

- **Resource allocator** – manages and allocates resources.
 - Resources: CPU time, Memory Space, file storage space, I/O devices and son on.
- **Control program** – controls the execution of user programs and operations of I/O devices .
- **Kernel** – the one program running at all times (all else being application programs).
- The two goals, efficiency and convenience are sometimes contradictory
- Much of OS theory is concentrates on optimal use of resources.

Sources of Complexity

- “Complex” means “difficult to understand”
- Five signs of complexity
 - Large number of components
 - More components means more complex
 - Large number of inter-connections
 - More interconnections mean more complex
 - Many irregularities
 - Lack of regularity, non—non-repetitive interconnections, and exceptions means more complex.
 - A long description
 - The complexity of the computational objects is the length of the description.
 - A team of designers, implementers or maintainers
 - If more people are required to maintain the system, it means more complex

Sources of Complexity:

Cascading and interacting Requirements

- More requirements, more complexity
- As they increase, the interactions increases
- Example: telephone
 - Call waiting, call return, call forwarding, call blocking, reverse billing, caller ID, Caller ID blocking, anonymous call rejection, do not disturb and so on
- Principle of escalating complexity
 - Adding a requirement increases complexity out of proportion

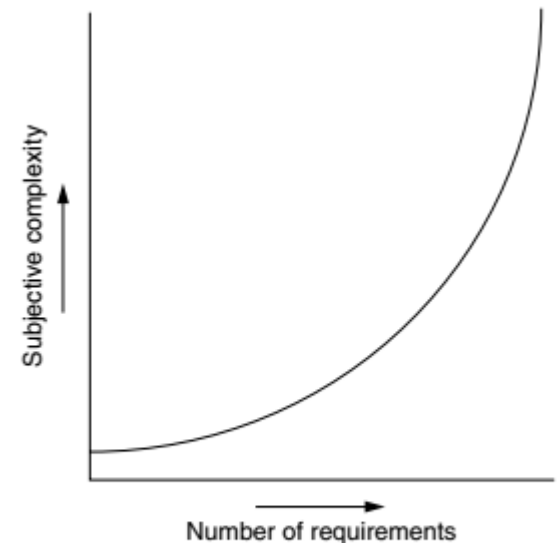


FIGURE 1.1

The principle of escalating complexity.

Sources of Complexity:

Generality

- Generality:
 - Meeting many requirements with single design
 - Applying to a variety of circumstances
- Use good judgment how much generality is required
 - Automobile with four independent steering wheels
 - Increases complexity
 - Designing a vehicle one can drive on the high way and use as a boat
 - May not be efficient
- Avoid excessive generality
 - If it is good for everything, it is good for nothing.

Sources of Complexity: Scaling

- Scaling increases complexity
 - Example 1
 - Small insects absorb oxygen through their skins
 - Large organisms require skins in proportion to the cube of their size
 - Addition of lungs and blood vessels
 - Example 2
 - Programmer of four-bit microprocessor can write code to control a toaster in binary language
 - Programmers of video games on 64-bit microprocessors require computers, operating system compilers, video editors, special effect generators, and so on

Sources of Complexity: Scaling...

- When the environment changes, requirements change
 - Knowledge of how to maintain old equipment may be disappearing
- Original design may lose its relevance.
- Unanticipated requirements
 - Changes to the existing system increase propagation of effects
 - A bug introduces another bug
- Bottom-line
 - As systems age, they tend to accumulate more changes and make them more complex.

Sources of Complexity: Maintaining high utilization

- Desire is to get high performance or high efficiency
- Utilization should be high for a scarce resource
- If we try to increase the utilization of the limited resource, complexity increases.

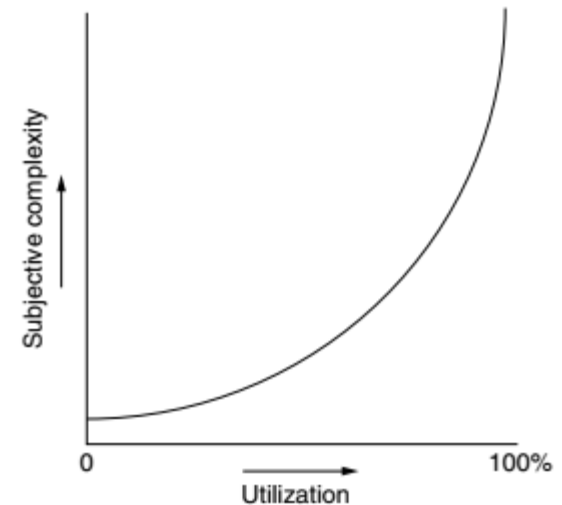


FIGURE 1.2

An example of diminishing returns: complexity grows with increasing utilization.

Coping with complexity I

- Modularity
- Abstraction
- Layering
- Hierarchy

Coping with complexity I

Modularity

- Divide-and-conquer technique
 - Analyse or design the system as a collection of interacting subsystems called modules
 - One can think about each module in an isolated manner.
- N statements; N bugs
- Time to fix a bug is proportional to the program.
 - Debugging time $= N * N$
- Dividing into K modules
 - $(N/K) * (N/K) * K$
 - (Approx.) $(N/K) * (N)$
- Reduces the debugging time by a factor of K.
- The property of modularity is universal.

Modularity..

- Allows incremental improvement
 - Easy to replace an inferior module with an improved one.
- It helps to address the complexity caused due to change.

Coping with complexity I

Abstraction

- The discovery of bug should lead to examining only one module
- There should be little or no propagation effects from one module to another.
 - Several ways to modularize the system
 - Some ways prove better than others.
- Abstraction
 - The ability of any module to treat all others entirely based on external specifications without the need for knowledge about what goes inside.
 - Examples:
 - Finding a compatible DVD player for a television set
 - Overnight pocket delivery service
- Well-designed and properly enforced modular abstractions are essentially important in limiting the impact of faults, and they control the propagation of effects

Coping with complexity I:

Robustness principle

- Be tolerant of inputs and strict on outputs
 - Key idea of modern mass production
- The robustness principle plays major role in computer systems.
 - Human interfaces, network protocols, and fault tolerance
- The Safety margin principle
 - Inputs should be within the range
 - Keep track of input ranges and report out-of-tolerance ranges.

Coping with complexity I: Layering

- Good abstraction minimizes the number of interconnections
- A way to reduce interconnections is the method of module organization called “layering”
- Mechanism
 - Set of mechanisms to use that are already complete (lower layer) to create a different complete set of mechanisms. (upper layer)
 - A layer may itself be implemented as several modules
 - A layer only interacts with peer modules and modules of the lower layer and the next higher layer.

Coping with complexity I

Hierarchy

- (1) Start with a small group of modules and assemble them into a stable, self-contained subsystem with a well-defined interface.
- (2) Assemble a small group of subsystems to produce a larger subsystem. Go to (1).
- The result is a tree structure known as a hierarchy.
- Hierarchy reduces the number of interconnections.

Summary

- Modularity, abstraction, layering, and hierarchy provide ways to divide things up and place the relevant modules in suitable relation to one another.

About computer systems

- Common problems with all complex systems
 - Emerging properties, propagation of effects, incommensurate scaling and trade-offs
 - Examples: space station design, management of economy, communication satellites, design of computer systems
- However, computer systems are different from other systems in two different ways
 - Physical laws do not limit the complexity. The rate of change of computer system technology is unprecedented

Computer Systems have no nearby bounds on composition

- They are digital and controlled by software
- Not limited by physical laws
- Analog system
 - Have engineering limitations. Each component contributes to noise.
 - Vibration of electromagnetic radiation
 - Component physical behavior changes
 - Noise from individual components accumulates
- As the number of components increases, at some point in time, noise will dominate the behavior of the system.
 - Natural biological, thermodynamic, macro-economic systems, and so on
 - Photocopy of the photocopy is not the same.

Digital systems are noise-free

- Digital logic
 - Range of analog values represents 0/1
 - Gate, flip-flop, memory chip, processor or computer system
- When 0 becomes 1, it leads to a big mistake, and big mistakes are easy to detect
- If a signal does not accumulate noise as it goes through a string of devices, noise does not limit the number of devices to be connected.
- Limit for growth: the ability of humans to understand them.
 - A processor chip contains two billion transistors

- Second reason for no nearby bounds is that computer systems are controlled by software.
- The contribution to complexity is worse.
- Hardware has physical limits--- speed of light
- Software appears to have no physical limits.
 - Typical operating system, database system has 10 million program statements.
- It is difficult to get the proper abstraction
- Leaking abstraction
 - Do not perfectly conceal the underlying implementation.
- Leakiness is like noise in analog systems

- It is easy for the designer to misuse the tools of modularity, abstraction, layering, and hierarchy to include some more complexity.
- New features keep getting added.

$d(\text{technology})/dt$ is unprecedented

- Cost for computation and communication dropped 30 percent each year.
- Some components have experienced a greater rate of improvement.
- Different between computer systems and other systems
 - $d(\text{technology})/dt$
- Complex systems take several years to build, they become obsolete by delivery time.
 - It is not the case with bridge, airplane, civil works

Coping with Complexity

- The **right modularity** from a sea of plausible alternative modularities
- The **right abstraction** from a sea of plausible alternative abstractions
- The **right layering** from a sea of plausible alternative layerings
- The **right hierarchy** from a sea of plausible alternative hierarchies

Coping with Complexity :

Iteration

- Start by building simple working system that meets only a modest number of requirements and evolve the system
 - Small steps will reduce the risk
- Release versions
- Successful iteration
 - Take small steps
 - Do not rush
 - Plan for feedback
 - Study about failures

Coping with Complexity II

Keep it simple

- It is one of the effective methods of handling complexity
- Designer must impose self-imposed limits
- Some of the reasons for adding new features
 - If new features are added, it will be more effective
 - Cost and performance are not factors
 - Each of new feature has been demonstrated some where
 - All exceptions are easy to deal
 - Competitors will market a system with new features
 - There is overconfidence among the designers
- The system designer should keep in mind about the cumulative impact of adding a new feature
- Another principle
 - Avoid sweeping simplifications

Outline

- Introduction
 - What is an Operating System?
- **Course topics and grading**

Objectives

- Traditionally, operating system concepts were developed for single processor (machine).
- Currently, shared memory, shared disk and shared-nothing architectures (computer network) are being employed for building computing systems.
 - Exploiting multiple processor cores and commodity machines to manage diverse computing loads, such as search engine, weather forecasting and mobile-based services.
- The objectives of this course to study the concepts (core as well as recent), which were evolved for building modern operating systems.

Objectives

- Understand the operational part of any computer.
 - To study the concepts (core as well as recent), which were evolved for building modern operating systems.
- Understanding the general principles of OS design.
 - Focus on general-purpose, multi-user systems.
 - Emphasis on widely applicable concepts rather than any specific features of any specific OS.
- Understanding problems, solutions and design choices.
- Understanding the structure of specific OSs: UNIX, LINUX, Solaris, WINDOWS 11, Android

Course topics

- Unit 1: Introduction, Computer System Hardware Review, Operating System Structures (9 hours)
- Unit 2: Process and thread management, CPU scheduling, Process Synchronization, Deadlocks (12 hours);
- Unit 3: Overview of Computer Networking (6 hours)
- Unit 4: Memory management, Virtual memory (6 hours);
- Unit 5: Basics of file systems, RAID and Storage Structures, Hadoop Distributed File System, Map Reduce, Cloud Computing: models and technologies, Virtual Machines. (10 hours)

CS3.304: Advanced Operating Systems – Class Schedule (Monsoon 2026)

class	Date	Topic	Lab
1	30/07/2026 (THU)	Introduction	
2	03/8/2026 (MON)	History and Background	Lab 1
3	06/8/2026 (THU)	Computer architecture	
4	10/8/2026 (MON)	System structures-1	
5	13/8/2026 (THU)	System structures-II	
6	17/8/2026 (MON)	Process concept	Lab 2
7	20/8/2026 (THU)	Inter-process Communication	
8	24/8/2026 (MON)	Multithreaded Programming	
9	27/8/2026 (THU)	QUIZ1	
10	31/8/2026 (MON)	Processor scheduling-I	Lab 3
11	03/9/2026 (THU)	Process scheduling-II	
12	07/9/2026 (THU)	Process Synchronization-I	
13	10/9/2026 (MON)	Process Synchoronization-II	
14	17/9/2026(THU)	Deadlocks	Lab 4
		Midterm Exam.	
15	28/9/2026 (MON)	Networking: Introduction and data link layer	
16	05/10/2026 (MON)	Network layer-1	
17	08/10/2026 (THU)	Network layer -II	
18	12/10/2026 (MON)	Transport layer	
19	15/10/2026 (THU)	Memory Management-I	
20	19/10/2026 (MON)	Memory Management-II	Lab 5
21	22/10/2026 (THU)	Quiz 2	
22	26/10/2025 (MON)	Virtual Memory-I	
23	29/10/2026 (THU)	Virtual memory-II	
24	02/11/2026 (MON)	Disk Scheduling, RAID	
25	05/11/2026 (THU)	Simple File system and map-reduce	
26	12/11/2026 (THU)	Distributed File System: Hadoop File System	
27	16/11/2026 (MON)	Cloud Computing: models and technologies	
28	19/11/2026 (THU)	Virtual machines	

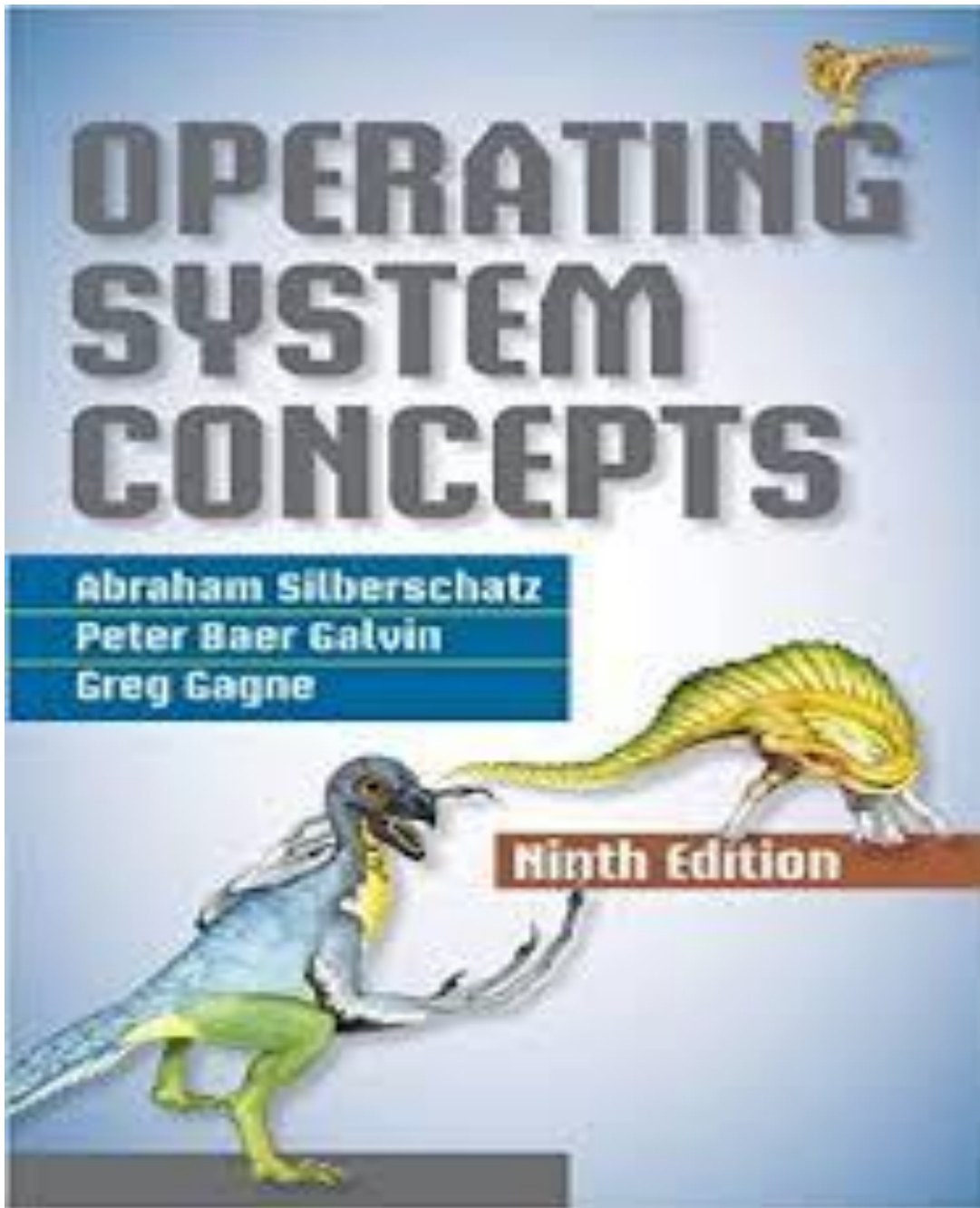
References

- Text books:
 - Silberschatzs, Operating System Concepts 10Ed Global Edition (Pb 2023)
 - Computer Networks, 6e by Tanenbaum (Author)
 - Distributed and Cloud Computing: From Parallel Processing to the Internet of Things Kai Hwang, Geoffrey C. Fox, Jack J. Dongarra, Elsevier, 2012
- Other References
 - Operating Systems, 9e by William Stallings (Author)
 - Charles Crowley, Operating Systems: A design-oriented approach, Tata McGraw-Hill, 1997.
 - Modern Operating Systems, 5th Edition, Tanenbaum (Author)
 - Research Papers

OPERATING SYSTEM CONCEPTS

Abraham Silberschatz
Peter Baer Galvin
Greg Gagne

Ninth Edition



FIFTH EDITION
COMPUTER NETWORKS



Distributed and Cloud Computing

From Parallel Processing to the Internet of Things

Kai Hwang • Geoffrey C. Fox • Jack J. Dongarra

MK
MORGAN KAUWMAN

Related Research Papers

1. Edsger W. Dijkstra, My Recollections of Operating System Design, by Dijkstra, ACM SIGOPS Operating Systems Review, Volume 39, Issue 2
2. D. M. Ritchie and K. Thompson, The Unix time-sharing system. Communications of the ACM 17, 7 (July 1974), 365–375.
3. Brown, Denning and Tichy, "Advanced Operating Systems," in Computer, vol. 17, no. 10, Oct. 1984.
4. Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, Andrew Warfield, Xen and the art of virtualization, SOSP '03: Proceedings of the nineteenth ACM symposium on Operating systems principles, Pages 164 – 177.
5. Ken Thompson, UNIX implementation. February 2002; Bell System Technical Journal 57(6). 2002; 57(6).
6. Ashvin Goel, Luca Abeni, Charles Krasie, Jim Snow, Jonathan Walpole, Supporting time-sensitive applications on a commodity OS, ACM SIGOPS Operating Systems Review, Volume 36, Pages 165 – 180
7. Andrew Baumann, Paul Barham, Pierre-Évariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, Akhilesh Singhanian: The multikernel: a new OS architecture for scalable multicore systems. SOSP 2009: 29-44
8. Jeffrey Dean, Sanjay Ghemawat: MapReduce: Simplified Data Processing on Large Clusters. OSDI 2004: 137-150
9. Sanjay Ghemawat, Howard Gobioff, Shun-Tak Leung: The Google file system. SOSP 2003: 29-43
10. Ivanilton Polato, Reginaldo Ré, Alfredo Goldman, Fabio Kon: A comprehensive view of Hadoop research - A systematic literature review. J. Netw. Comput. Appl. 46: 1-25 (2014)
11. Jeremy Andrus, Christoffer Dall, Alexander Van't Hof, Oren Laadan, Jason Nieh: Cells: a virtual mobile smartphone architecture. SOSP 2011: 173-187
12. Philip Alexander Levis, Samuel Madden, Joseph Polastre, Robert Szewczyk, Kamin Whitehouse, Alec Woo, David Gay, Jason L. Hill, Matt Welsh, Eric A. Brewer, David E. Culler: TinyOS: An Operating System for Sensor Networks. Ambient Intelligence 2005: 115-148

LAB WORK

- Five mini projects related to the above syllabus will be done by students in the laboratory
- Experiments will be on the exposing the working of several system calls of LINUX OS involving single and multiple real/virtual machines.
 - Installation; reversing a file; Shell writing
 - Process communication
 - Bounded buffer,
 - semaphores,
 - shared memory,
 - threads;
 - Replace "ls" with lookup;
 - Command line for /proc;
 - Memory management
- Note: The lab is very intensive. Please do not ask for the extension of deadline. Each experiment will be evaluated.

COURSE OUTCOMES

- After completion of this course successfully, the students will be able to,
 - CO-1. Explain the concepts of several modern computer operating systems (SOLARIS, LINUX, WINDOWS, MAC, Adroid,...)
 - CO-2: Implement the task on the top of given operating system, in an efficient manner, based on process and thread framework.
 - CO-3. Prescribe the appropriate scheduling/synchronization/memory management/virtual memory/protection for a given application.
 - CO-4. Architect the new application by selecting the appropriate system calls of the given operating system services.
 - CO-5. Develop a distributed application on multi-processor machine or multiple commodity processors connected through a network.

GRADING

Type of Evaluation	Weightage (in %)
Class room test 1	5%
Mid Sem Exam	15%
Class room test 2	5%
End Sem Exam	40%
Attendance	5%
Lab Assignments (mini projects)	30%

Lab Assignments

- You will be given the lab assignments in advance
 - The instructor will provide you with adequate background information to do the assignment
 - Some installations will be necessary. Install well in advance and if you encounter any problems with any installation, you can contact your TA
- Policies for lab
 - You should try to solve any programming issues by yourself first, remember that troubleshooting is how you would actually be learning a lot
 - Every time you encounter any minor programming problem, if you ask the TA or instructor, you will lose out on an excellent learning experience
 - Only if you are unable to solve the problem after putting in a reasonable amount of effort, you should contact your TA or your instructor
 - You should also look online for solutions to your problems, but do not copy code from anywhere
- Grading criteria
 - The overall quality of your lab assignment submission in terms of correctness, quality of code, system design etc.

Plagiarism

- This course has a zero-tolerance policy w.r.t. plagiarism
- Any instance of plagiarism will result in serious penalties (e.g., an F grade for the entire course, among other penalties)
- Forget about doing any kind of plagiarism

Deadlines

- Strict deadlines: You will not be able to submit after the deadline has already passed.

Accessing the Course Materials

- The presentations, the materials, lab assignments are available on the course portal
 - Pls. access the course portal regularly
- All students should procure two books, as soon as possible
 - Silberschatz, A, Galvin, P, Gagne, G. Operating System Concepts, Addison-Wesley (8th or latest edition).
 - Computer Networks (5th Edition or latest edition) Andrew S. Tanenbaum, David J. Wetherall Prentice Hall.

Asking Questions

- Theory and lab related questions can be asked through the course portal
 - It is better, because all other students can gain the related knowledge
 - Try to ask as many subject/lab questions as possible.
 - Do not hesitate to ask silly/simple questions regarding lab or theory
 - And try to get the problem resolved as soon as possible

Maximize the benefit from the course

- In the course, we are going to study about how a wonderful artifact is being built through the efforts of thousands of researchers
 - Operating system has become part and parcel of every tool
 - Microsoft windows has **50 million** lines of code.
- Focus on a thorough understanding of the concepts, not on memorizing
 - Understand every sentence of the book

Cone of Learning (Edgar Dale)

After 2 weeks
we tend to remember...

10% of what we read

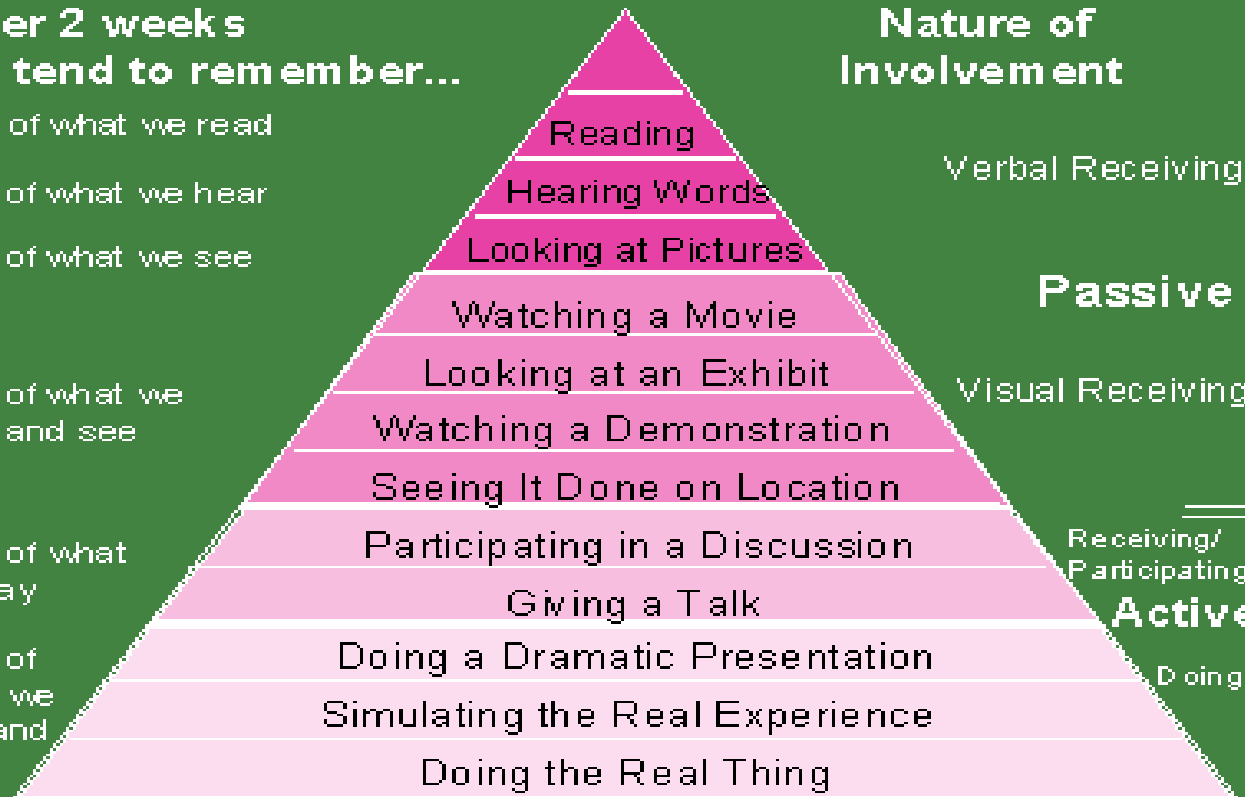
20% of what we hear

30% of what we see

50% of what we
hear and see

70% of what
we say

90% of
what we
say and
do



Edgar Dale , *Audio-Visual Methods in Teaching* (3rd Ed.) , Holt, Rinehart, and Winston (1969).

Source: <http://www.cals.ncsu.edu/agexed/sae/ppt1/sld012.htm>

Cone of Learning (Edgar Dale)

After 2 weeks
we tend to remember...

10% of what we read

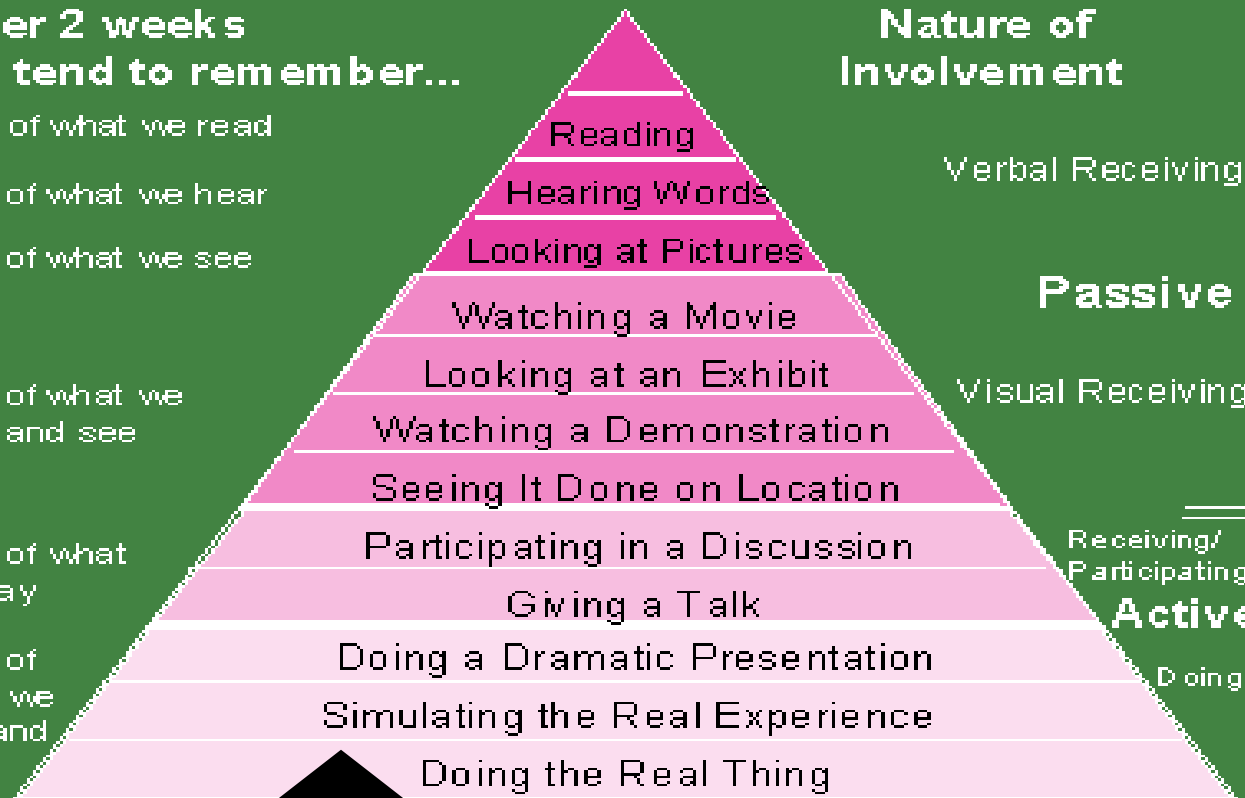
20% of what we hear

30% of what we see

50% of what we
hear and see

70% of what
we say

90% of
what we
say and
do



Edgar Dale, *Analog-*

Methods in Teaching (3rd Ed.), Holt, Rinehart, and Winston (1969).

Bottomline: Do the assignments sincerely because it will facilitate you in **INTERNALIZING the ideas/techniques you learnt in this course.**

Source: <http://www.cals.ncsu.edu/agexed/sae/ppt1/sld012.htm>